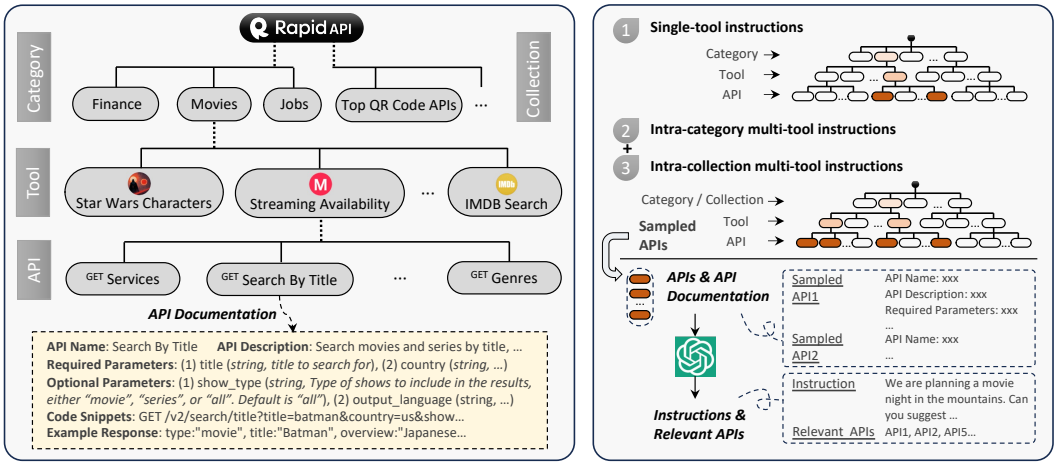

Figure 3: The hierarchy of RapidAPI (left) and the process of instruction generation (right).

2.1 API Collection

We start by introducing RapidAPI and its hierarchy, followed by how we crawl and filter APIs.

RapidAPI Hub RapidAPI is a leading API marketplace that connects developers with thousands of real-world APIs, streamlining the process of integrating diverse services into applications. Developers can test and connect with various APIs by registering only a RapidAPI key. All APIs in RapidAPI can be classified into 49 *coarse-grained* **categories** (link), such as sports, finance, and weather. The categories associate an API with the most relevant topic. Additionally, the hub also provides 500+ *fine-grained* categorization called **collections** (link), e.g., Chinese APIs and database APIs. APIs in the same collection share a common characteristic and often have similar functionalities or goals.

Hierarchy of RapidAPI As shown in Figure 3, each tool may be composed of multiple APIs. For each tool, we crawl the following information: the name and description of the tool, the URL of the host, and all the available APIs belonging to the tool; for each API, we record its name, description, HTTP method, required parameters, optional parameters, request body, executable code snippets for API call, and an example API call response. This rich and detailed metadata serves as a valuable resource for LLMs to understand and effectively use the APIs, even in a zero-shot manner.

API Filtering Initially, we gathered $10,853$ tools ($53,190$ APIs) from RapidAPI. However, the quality and reliability of these APIs can vary significantly. In particular, some APIs may not be well-maintained, such as returning 404 errors or other internal errors. To this end, we perform a rigorous filtering process (details in appendix A.1) to ensure that the ultimate tool set of ToolBench is reliable and functional. Finally, we only retain $3,451$ high-quality tools ($16,464$ APIs).

2.2 Instruction Generation

Different from prior works, we specifically focus on two crucial aspects for instruction generation: (1) **diversity**: to train LLMs to handle a wide range of API usage scenarios, thereby boosting their generalizability and robustness; and (2) **multi-tool usage**: to mirror real-world situations that often demand the interplay of multiple tools, improving the practical applicability and flexibility of LLMs. To this end, instead of brainstorming instructions from scratch and then searching for relevant APIs, we sample different combinations of APIs and craft various instructions that involve them.

Generating Instructions for APIs Define the total API set as $\mathbb{S}_{\text{API}}$, at each time, we sample a few APIs: $\mathbb{S}_{\text{N}}^{\text{sub}} = \{\text{API}_1, \cdots, \text{API}_{\text{N}}\}$ from $\mathbb{S}_{\text{API}}$. We prompt ChatGPT to understand the functionalities of these APIs and then generate (1) possible instructions (Inst_*) that involve APIs in $\mathbb{S}_{\text{N}}^{\text{sub}}$, and (2) relevant APIs ($\mathbb{S}_*^{\text{rel}} \subset \mathbb{S}_{\text{N}}^{\text{sub}}$) for each instruction ($\text{Inst}_*$), i.e., $\{[\mathbb{S}_1^{\text{rel}}, \text{Inst}_1], \cdots, [\mathbb{S}_{\text{N}'}^{\text{rel}}, \text{Inst}_{\text{N}'}]\}$, where N' denotes the number of generated instances. These (instruction, relevant API) pairs will be used for

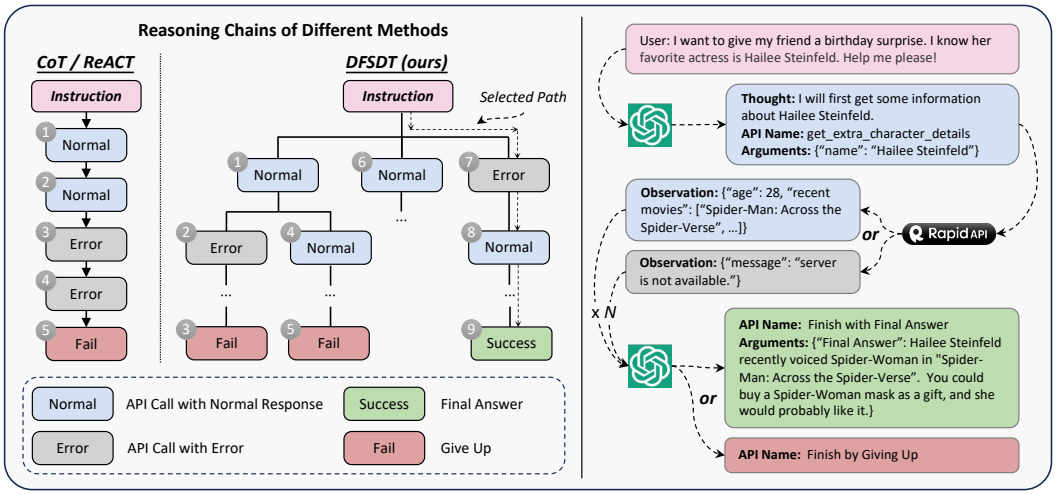

Figure 4: A comparison of our DFSDT and conventional CoT or ReACT during model reasoning (left). We show part of the solution path annotation process using ChatGPT (right).

training the API retriever in § 3.1. We use different sampling strategies (introduced later) to cover all APIs and most of their combinations, thus ensuring the diversity of our instructions.

The prompt for ChatGPT is composed of (1) a general description of the intended instruction generation task, (2) comprehensive documentation of each API in $\mathbb{S}_{\text{N}}^{\text{sub}}$, which helps ChatGPT understand their functionality and interplay, and (3) three in-context seed examples $\{\text{seed}_1, \text{seed}_2, \text{seed}_3\}$. Each seed example is an ideal instruction generation written by human experts. These seed examples are leveraged to better regulate ChatGPT's behavior through in-context learning. In total, we wrote 12 / 36 diverse seed examples ($\mathbb{S}_{\text{seed}}$) for the single-tool / multi-tool setting, and randomly sampled three examples at each time. Detailed prompts for instruction generation are described in appendix A.7. Overall, the generation process can be formulated as follows:

$$\underset{\{\text{API}_1,\cdots,\text{API}_\text{N}\}\in\mathbb{S}_{\text{API}},\{\text{seed}_1,\cdots,\text{seed}_3\}\in\mathbb{S}_{\text{seed}}}{\text{ChatGPT}}(\{[\mathbb{S}_1^{\text{rel}}, \text{Inst}_1], \cdots, [\mathbb{S}_{\text{N}'}^{\text{rel}}, \text{Inst}_{\text{N}'}]\}|\text{API}_1, \cdots, \text{API}_\text{N}, \text{seed}_1, \cdots, \text{seed}_3).$$

Sampling Strategies for Different Scenarios As shown in Figure 3, for the **single-tool instructions (I1)**, we iterate over each tool and generate instructions for its APIs. However, for the **multi-tool setting**, since the interconnections among different tools in RapidAPI are sparse, random sampling tool combinations from the whole tool set often leads to a series of irrelevant tools that cannot be covered by a single instruction in a natural way. To address the sparsity issue, we leverage the RapidAPI hierarchy information. Since tools belonging to the same RapidAPI *category* or *collection* are generally related to each other in the functionality and goals, we randomly select 2-5 tools from the same category / collection and sample at most 3 APIs from each tool to generate the instructions. We denote the generated instructions as **intra-category multi-tool instructions (I2)** and **intra-collection multi-tool instructions (I3)**, respectively. Through rigorous human evaluation, we find that instructions generated in this way already have a high diversity that covers various practical scenarios. We also provide visualization for instructions using Atlas (link) to support our claim.

After generating the initial set of instructions, we further filter those with the hallucinated relevant APIs by assessing whether they exist in $\mathbb{S}_{\text{N}}^{\text{sub}}$. Finally, we collect nearly 200k qualified (instruction, relevant API) pairs, including 87413, 84815, and 25251 instances for I1, I2, and I3, respectively.

2.3 Solution Path Annotation

As shown in Figure 4, given an instruction Inst_*, we prompt ChatGPT to search for a valid action sequence: $\{a_1, \cdots, a_\text{N}\}$. Such a multi-step decision-making process is cast as a multi-round conversation for ChatGPT. At each round t, the model generates an action a_t based on previous interactions, i.e., $\text{ChatGPT}(a_t|\{a_1, r_1, \cdots, a_{t-1}, r_{t-1}\}, \text{Inst}_*)$, where r_* denotes the real API response. For each

a_t, ChatGPT should specify its "thought", which API to use, and the specific parameters for this API, i.e., a_t has the following format: "`Thought:` $\cdots$, `API Name:` $\cdots$, `Parameters:` $\cdots$".

To leverage the **function call** feature of ChatGPT, we treat each API as a special function and feed its API documentation into ChatGPT's function field. In this way, the model understands how to call the API. For each instruction Inst_*, we feed all the sampled APIs $\mathbb{S}_{\text{N}}^{\text{sub}}$ to ChatGPT's as available functions. To let ChatGPT finish an action sequence, we define two additional functions, i.e., "Finish with Final Answer" and "Finish by Giving Up". The former function has a parameter that corresponds to a detailed final answer to the original instruction; while the latter function is designed for cases where the provided APIs cannot complete the original instruction after multiple API call attempts.

Depth First Search-based Decision Tree In our pilot studies, we find that CoT (Wei et al., 2023) or ReACT (Yao et al., 2022) has inherent limitations: (1) **error propagation**: a mistaken action may propagate the errors further and cause the model to be trapped in a faulty loop, such as continually calling an API in a wrong way or hallucinating APIs; (2) **limited exploration**: CoT or ReACT only explores one possible direction, leading to limited exploration of the whole action space. Hence even GPT-4 often fails to find a valid solution path, making annotation difficult.

To this end, we propose to construct a decision tree to expand the search space and increase the possibility of finding a valid path. As depicted in Figure 4, our DFSDT allows the model to assess different reasoning paths and choose to either (1) proceed along a promising path or (2) abandon an existing node by calling the "Finish by Giving Up" function and expand a new node. During node expansion, to diversify the child nodes and expand the search space, we prompt ChatGPT with the information of the previously generated nodes and explicitly encourage the model to generate a distinct node. For the searching process, we prefer depth-first search (DFS) instead of breadth-first search (BFS) because the annotation can be finished as long as one valid path is found. Using BFS will cost excessive OpenAI API calls. More details are described in appendix A.8. We perform DFSDT for all the generated instructions and only retain those passed solution paths. Ultimately, we generate $126,486$ (instruction, solution path) pairs, which are used to train ToolLLaMA in § 3.2.

3 EXPERIMENTS

In this section, we investigate the performance of ToolLLM framework. We first introduce the evaluation metric and evaluate the efficacy of API retriever and DFSDT in § 3.1. Then we present the main experiments in § 3.2, followed by a generalization experiment in § 3.3.

3.1 PRELIMINARY EXPERIMENTS

ToolEval Considering the API's temporal variability on RapidAPI and the infinite potential solution paths for an instruction, it is infeasible to annotate a fixed ground-truth solution path for each test instruction. Moreover, when comparing different models, it is crucial to ensure they employ the same version of APIs during evaluation. Considering that human evaluation can be time-consuming, we follow AlpacaEval (Li et al., 2023b) to develop an efficient evaluator **ToolEval** based on ChatGPT, which incorporates two evaluation metrics (details in appendix A.5): (1) **Pass Rate**: it calculates the proportion of successfully completing an instruction within limited budgets. The metric measures the executability of instructions for an LLM and can be seen as a basic requirement for ideal tool use; and (2) **Win Rate**: we provide an instruction and two solution paths to ChatGPT evaluator and obtain its preference (i.e., which one is better). We pre-define a set of criteria for both metrics and these criteria are organized as prompts for our ChatGPT evaluator. We evaluate multiple times based on ChatGPT to improve the reliability. Then we calculate the average results from the evaluator.

Through rigorous testing (details in appendix A.5), we find that ToolEval demonstrates a high agreement of 87.1% in pass rate and 80.3% in win rate with human annotators. This shows that ToolEval can reflect and represent human evaluation to a large extent.

Efficacy of API Retriever The API retriever aims to retrieve relevant APIs to an instruction. We employ Sentence-BERT (Reimers & Gurevych, 2019) to train a dense retriever based on BERT-BASE (Devlin et al., 2019). The API retriever encodes the instruction and API document into two embeddings, and calculates their relevance with embedding similarity. For training, we regard the relevant APIs of each instruction generated in § 2.2 as positive examples and sample a few other